Credit Card Fraud Detection System

Project Evaluation, Milestone Verification & Progress Report

Phase III / Mid-Term Review
Working Prototype (~90%)

EVALUATION STAGE	TARGET DOMAIN	SUBMISSION DEADLINE	OVERALL PROGRESS
Phase III / Mid-Term	Financial Fraud & ML	29 October 2026	~90% Functional

📌 **Critical Submission Checkpoints & Verification**

- **Weekly Progress Reports (WPRs):** 3 WPRs submitted to supervisor (WPR-1: Dataset EDA & Ingestion; WPR-2: Model Benchmarking & SMOTE; WPR-3: UI, Batch Ingestion & Session State).
- **Project Guide Verification:** System features and architecture reviewed with project guide.
- **Live Demonstration:** Streamlit web application active and verifiable on local test environment (Port 8501).

1 Select Evaluation Stage

Stage Selected: **Phase III Review / Mid-Term Evaluation**

The project has achieved Phase III maturity with an integrated, end-to-end working prototype that fulfills core algorithmic, architectural, and user-interaction requirements.

2 Current Implementation / Prototype Status

The system is a **fully functional end-to-end working software prototype (~90% complete)** featuring an interactive web dashboard, real-time prediction engine, database persistence, and analytical reporting.

- **Frontend & State:** Multi-page Streamlit dashboard with custom Bonsai/Green styling, dark/light themes, and client-side session persistence.
- **Trained Models:** Pre-trained pipelines (Random Forest, XGBoost, Decision Tree, Logistic Regression) stored as production `.pkl` artifacts.
- **Inference Modes:** Real-time single transaction scoring and bulk batch CSV processing with instant risk categorization.

3 Implementation Completed

Authentication & Role-Based Access
Bcrypt password hashing, SQLite persistence, Admin vs. Analyst privileges, activity audit trails, and automatic session restoration across browser reloads.

Data Pipeline & Imbalance Handling
Ingestion of European credit card transactions (284,807 instances), standard scaling for Time and Amount, and SMOTE oversampling for balanced training.

Machine Learning Engine
Training and persistence of 4 classification models with hyperparameter tuning, evaluated using Precision-Recall and AUC-ROC curves.

Interactive Dashboard & Analytics
Interactive Plotly visualization for Confusion Matrices, Feature Importance, Risk Level distribution, and exportable CSV audit logs.

4 Code / Hardware / System Integration Status

Layer / Component	Technology	Integration Status
User Interface	Streamlit, HTML5, Custom CSS	**Fully Integrated** Seamless view switching & sidebar toggle
ML Inference Engine	Scikit-learn, XGBoost, Joblib	**Fully Integrated** Serialized models loaded on application startup
Database & Storage	SQLite3, JSON metrics store	**Fully Integrated** User authentication, audit logs & model metrics
Session Bridge	JavaScript localStorage Bridge	**Fully Integrated** Sessions persist reliably across page refreshes

5 & 10 Testing, Validation & Experimental Performance Results

Models were evaluated on a hold-out test set of **56,962 transactions** (including 98 fraudulent transactions) after training on 454,902 SMOTE-balanced samples:

Model	Accuracy	Recall (Fraud)	Precision	F1-Score	AUC-ROC	False Positives
Random Forest (Best)	**99.90%**	**84.69%**	**67.48%**	**75.11%**	**97.38%**	**40 / 56,864**
XGBoost	99.69%	86.73%	34.55%	49.42%	97.60%	161 / 56,864
Logistic Regression	97.43%	91.84%	5.81%	10.94%	96.98%	1,458 / 56,864
Decision Tree	98.35%	80.61%	7.91%	14.40%	89.50%	920 / 56,864

Key Finding: Random Forest proved superior for enterprise deployment due to high precision (67.48%) and an extremely low false-positive rate (0.07%), minimizing customer friction while maintaining 84.7% fraud detection.

6 Current Milestone Completion Status

Milestone Phase	Scope	Progress
Phase I: Research & EDA	Dataset acquisition, distribution analysis, PCA inspection	**100% Completed**
Phase II: Model Benchmarking	SMOTE pipeline, algorithm comparison, hyperparameter tuning	**100% Completed**
Phase III: System Integration	Streamlit UI, Auth, Batch Predictor, Session persistence	**95% Completed**
Phase IV: Deployment & Packaging	Dockerization & cloud deployment	**40% In Progress**

7 & 12 Major Technical Challenges & Pending Work

Resolved: Extreme Class Imbalance
Fraud accounted for only 0.172% of transactions. Solved using **SMOTE oversampling**, which balanced the minority class during training without duplicating entries, lifting fraud recall from 58% to 84.7%.

Resolved: Streamlit Session Loss on Refresh
Streamlit re-executes Python scripts from scratch on refresh, logging users out. Solved by implementing a **parent-frame localStorage bridge** and query-parameter rehydration.

Remaining Tasks for Final Submission:

- Containerize application with Docker for seamless cross-platform deployment.
- Integrate SHAP / LIME explainability widgets to show exact feature contributions for flagged transactions.
- Perform high-throughput stress testing with 10,000+ transaction batch feeds.

8, 9 & 11 Architecture, Validation & Supporting Evidence

- **Architecture Pattern:** Modular 3-Tier Architecture (Presentation Layer via Streamlit ↔ Service & Inference Layer via Scikit-Learn/XGBoost ↔ Data Layer via SQLite & Serialized Models).
- **Validation Strategy:** Stratified hold-out split (80/20) ensuring identical fraud ratios across train and test sets with zero data leakage.
- **Code Repository:** Structured Git repository with modular separation: `src/` (engines), `views/` (pages), `models/` (artifacts), and `data/`.
- **Demonstration Verification:** Ready for live localhost evaluation by supervisor and external examiner.